

# Autonomous Farm Lane Navigation and Plant Counting Using RGB-D Perception

## Technical Interview Assignment

### Aim

Build an autonomous navigation system for a Husky UGV that drives through irregular, non-uniform farm lanes using a single RGB-D camera, while detecting and counting the plants it encounters along the way.

### Problem Statement

You are given a Gazebo simulation of a farm with irregularly angled, non-parallel lanes and a distribution of plants. The Husky robot starts at a marked **blue location** and must autonomously reach a marked **green goal location**, using only one Intel RealSense RGB-D camera for all perception. No LiDAR, additional cameras, GPS, or other localization sensors are permitted.

Using this camera, your system must:

1. Perceive the lane layout well enough to plan a safe path, without relying on color/HSV thresholding.
2. Navigate from start to goal using ROS 2 Humble and Nav2, planning and executing the route in real time rather than along a hardcoded trajectory.
3. Detect and count the plants visible during the run, without double-counting a plant seen from multiple angles.
4. Record and visualize the run — camera feed, detections, and the path actually driven.

You will also be asked to explain your design choices, including which parts of the pipeline are learned versus hand-engineered.

### Resources

Provided to you:

- A Gazebo `.world` file containing the farm layout, lane geometry, plant placement, and the blue (start) and green (goal) markers.

**You are expected to source/configure:**

- A ROS 2-compatible Husky UGV model (URDF/Xacro) and its Gazebo integration, from open-source packages.
- An Intel RealSense RGB-D camera URDF/plugin and the corresponding ROS 2 driver, configured for RGB stream, depth stream, and camera intrinsics, all exposed as ROS 2 topics.

**Required software stack:**

- ROS 2 Humble
- Gazebo
- Nav2
- RViz

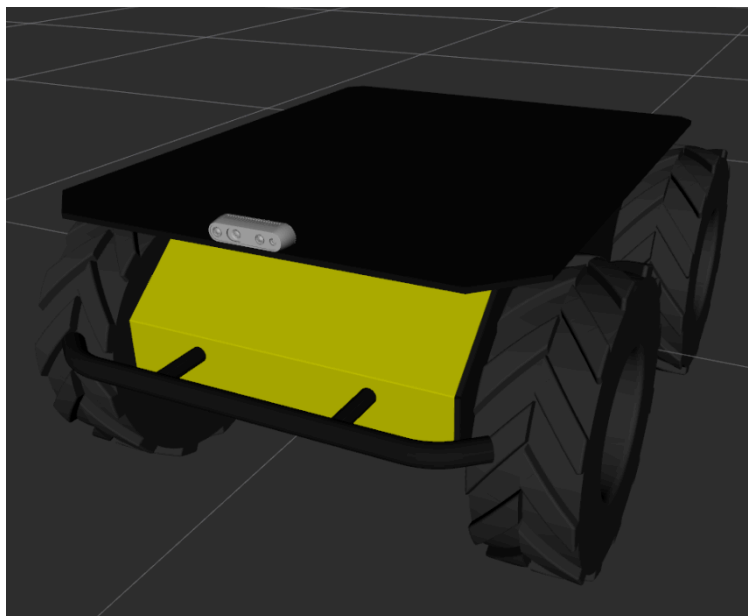
## Robot and Sensor Configuration

### Robot Platform

Husky UGV, modeled as a ROS 2-compatible URDF/Xacro, integrated into Gazebo.

### Camera: Placement and Justification

A single Intel RealSense RGB-D camera is the only sensor allowed, and it must be mounted as follows:



This placement is intentional, not a constraint to work around is **not allowed**: one RealSense RGB-D camera (RGB stream + depth stream + intrinsics). **Not allowed**: LiDAR, additional cameras, GPS, or any other localization sensor.

## Simulation Environment

The provided world contains multiple farm lanes at irregular angles and non-uniform orientations, a set of plants distributed through the field, and two fixed markers (**Load the Asset in Gazebo**):

- **Blue marker** — robot start position.
- **Green marker** — navigation goal.

The start and goal positions must be read from the environment at runtime — they must **not** be hardcoded into your navigation logic.

## Functional Requirements

### 1. Farm Lane Detection

Detect navigable lanes from RGB-D data alone, robust to varying lane orientation, irregular geometry, and appearance changes.

**Disallowed (will result in disqualification)**: HSV thresholding, fixed RGB thresholding, manually tuned color segmentation, or green-pixel-based lane following.

**Acceptable directions**: learned semantic segmentation, RGB-D ground/lane estimation, geometric feature extraction, depth-based scene understanding, or another principled approach of your choosing.

### 2. Autonomous Navigation

Using ROS 2 Humble and Nav2, convert your perception output into navigation goals and a feasible, collision-free trajectory from the blue marker to the green marker.

### 3. Plant Detection and Counting

Detect plants from RGB-D input during navigation and maintain a running count, taking care not to recount the same plant when it's observed from more than one viewpoint.

### 4. Path Recording and Visualization

Record the full run and produce:

- **A video** showing the RGB camera feed, the detected lane region, detected plants, navigation status, and the final plant count.
- **A path visualization** overlaying the planned path and the actual trajectory on the map, showing the area the robot covered.

## Technical Explanation

Along with your submission, provide a short write-up covering:

- **Perception logic** — how lanes and plants are detected, and how RGB and depth are combined.
- **Navigation logic** — how navigation goals are generated and how movement decisions are made.
- **Hardcoded vs. learned components** — explicitly list any fixed thresholds, safety margins, geometric assumptions, or manually tuned parameters, and distinguish them from learned components (e.g., trained segmentation/detection models).

## Evaluation Metrics

| Metric                          | What it measures                                                                                        |
|---------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Navigation success</b>       | Robot reaches the green goal without navigation failure.                                                |
| <b>Lane coverage efficiency</b> | $(\text{Covered lane area} / \text{Total available lane area}) \times 100$ — higher is better.          |
| <b>Path efficiency</b>          | Distance traveled, deviation between planned and actual trajectory, and any unnecessary movement.       |
| <b>Plant counting accuracy</b>  | Estimated count vs. ground truth — including false detections, missed detections, and duplicate counts. |

## **Deliverables**

- Complete ROS 2 workspace with robot and camera integration.
- Demonstration video (perception, plant detection, navigation progress, trajectory visualization, final plant count).
- Technical write-up covering algorithm design, navigation logic, hardcoded assumptions, and performance evaluation against the metrics above.